

CS1910

Computer Science I

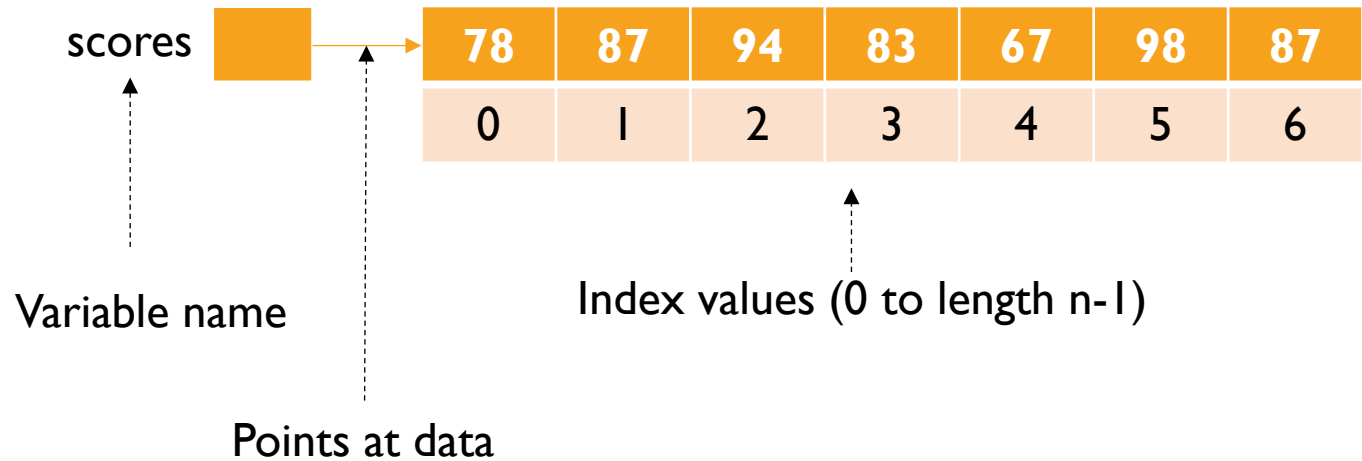
Lists

EXAMPLE CODE

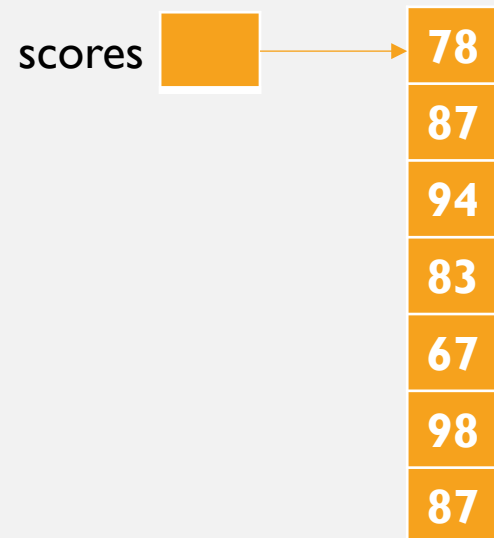
- <https://colab.research.google.com/github/cdspower-upei/2026W-CS1910-Students/blob/main/L11/CS1910-Lists-Examples.ipynb>

MUTABLE SEQUENCES

- Lists are sequences (like strings and ranges)
 - They represent ordered data
 - We can use indexing on sequences
 - Can hold items of different data types (even in the same list)
- Lists are mutable
 - The contents can be modified (without creating a new list)
 - Strings and ranges are immutable



VISUALIZATION OF LISTS



MUTABLE SEQUENCES

- Can change the indexed internal values of the list
- Indexed values can be used in expressions and function arguments like variables
- Can use the values directly or assign them to variables to be used separate from the list
- Variable values are separate copies of the value in the list – they can be modified separately if they are atomic types

```
01  # Expressions with list values
02  scores = [78, 87, 94, 83, 67, 98, 87]
03  sub_scores = scores[2:4] # slice index 2 and 3
04  mean = (scores[0] + scores[2])/2
05  print(mean) # 86.0
06  scores[2] = 5
07  mean = (scores[0] + scores[2])/2
08  print(mean) # 41.5
09  highest = scores[5]
10  lowest = scores[4]
11  scores[5] += 2 #100
12  highest += 1 #99
13  print("High: " + str(scores[5])) # High: 100
14  print(f"High Variable: {highest}") # Low: 99
15  print(f"Low: {scores[4]}") # Low: 67
16  print(f"Sub_scores: {sub_scores}") #Sub_scores: [94, 83]
```

CREATING LISTS

```
01  # Creating an empty list
02  my_list = []
03  # List with strings
04  fruits = ["apple", "banana", "cherry"] # ['apple', 'banana', 'cherry']
05  # List with integers
06  numbers = [1, 2, 3, 4, 5] # [1, 2, 3, 4, 5]
07  # List with mixed values
08  # Each value in each index has its own type
09  mixed_list = [10, "hello", 3.14, True] # [10, 'hello', 3.14, True]
10  # Convert a string to a list
11  title_list = list("Logicomix") # ['L', 'o', 'g', 'i', 'c', 'o', 'm', 'i', 'x']
12  # List with one value in it
13  small_list = [0] # [0]
14  # Ten zeros
15  zeros = [0] * 10 # [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
16  # Ten empty strings
17  empty_strings = [""] * 10 # ["", "", "", "", "", "", "", "", "", ""]
```

Not all programming languages allow this



ADDING TO LISTS

```
08 # Adding Items to Lists
09 # Creates an empty list
10 my_list = [] # []
11 # Mutation Methods for Adding to List
12 #Append - Add to empty list
13 my_list.append(1) # [1]
14 # Append - Add to end of list
15 my_list.append(23) # [1,23]
16 # Append - Add to end of list
17 my_list.append("!") # [1, 23, "!"]
18 # Insert - Add specific value at specific index
19 my_list.insert(1, "orange") #[1, "orange", 23, "!"]
20 # Insert - Add to the front of the list
21 my_list.insert(0, "banana") # ["banana", 1, "orange", 23, "!"]
22 # Insert - Add specific value at end of list specific index
23 # Equivalent to .append(23)
24 my_list.insert(5, 23) # ["banana", 1, "orange", 23, "!", 23]
25 # Insert - Add specific value well beyond the end of the list - goes to end of list
26 my_list.insert(99, "apple") # ["banana", 1, "orange", 23, "!", 23, "apple"]
```

REMOVAL FROM LISTS

- Many different ways to remove items from the list
 - `remove` – removes by value
 - `pop` – removes at index and returns the value for later use
 - `del` – removes item(s) from list but does not return anything to use later
 - `del` is a Python keyword not a method or function (thus no parentheses)
- Why are these all so different?
 - Function?
 - Method?
 - Keyword?

```
01 # Removing Items from Lists
02 # Remove the first instance of an item from the lists
03 my_list.remove(23) #["banana", 1, "orange", "!", 23, "apple"]
04 # Remove apple and store for later
05 last_item = my_list.pop() #["banana", 1, "orange", "!", 23]
06 print(last_item) # apple
07 # Pop - Optional parameter to remove index
08 # Remove first item banana for later
09 first_item = my_list.pop(0) #[1, "orange", "!", 23]
10 print(first_item) # banana
11 # Delete an index without knowing the value contained within
12 del my_list[0] #["orange", "!", 23]
13 # Delete a range of indices
14 del my_list[0:2] #Removes inclusive 0 and 1 - [23]
```


WHAT ARE OBJECTS?

- Up until strings, we always used functions as: *function(variable)*
 - Examples: `abs(-34)`, `max(3,4)` even then we used some functions `len("hello")`
- When we introduced strings, we had functions that could be accessed through a dot notation
 - Example: `my_string.upper()`
- That is because strings are what is called *Objects* in python and other languages
 - Objects are compound data types that have data that they maintain
 - Objects also have functions called *methods* that work on the data in the object
- Example: `my_string.upper()`
 - String object `my_string` has a set data in a sequence
 - `.upper()` is a method that only works on those characters that are the data contained `my_string`
- Why is there things like `len("hello")` instead of `my_string.length()`?
 - Python has predefined object methods that allow you to use those functions for consistency with atomic types
- NB: Objects are a key concept that will be important in CS1920 where you build your own types.
 - In CS1910 you just need to know that objects have data, and functions that work on that data

LIST METHODS

- Lists have many built-in methods
- Some useful methods
- `len(list)`: Returns the number of items in the list.
- `list.sort()`: Sorts the list in ascending order (in-place).
- `list.reverse()`: Reverses the order of items in the list (in-place).
- `list.count(item)`: Returns the number of times an item appears in the list.
- `list.index(item)`: Returns the index of the first occurrence of an item.
- `list.clear()`: Removes all items from the list.

```
1 my_list = [1, 2, 75, 3, 3, 4]
2 len(my_list) #length of the list
3 my_list.count(3) # count occurrences of 3 - 2
4 my_list.index(3) # index of first 3 - index 3
5 my_list.sort() # sorted list in place (same variable)
6 # sorted my_list = [1, 2, 3, 3, 4, 75]
7 my_list.index(3) # index of first 3 - index 2
8 my_list.reverse() # reverse list [75, 4, 3, 3, 2, 1]
9 my_list.clear() # List emptied - []
```

Technically integers (floats, bool) are objects but they are not mutable

ALIASING LISTS

- Mutable objects behave differently than immutable values like int and float
- They are passed around as object references – a pointer – rather than a true “copy” of a list
- Each is an *alias* for the other one
 - Like a secret identity for a superhero – both names refer to the same person

```
01 names_1 = ["Clark", "Diana", "Bruce"]
02 names_2 = names_1 # names_2 = ["Clark", "Diana", "Bruce"]
03 names_1[2] = "Oliver"
```

#Both lists are now ["Clark", "Diana", "Oliver"]



- After line 03 – Bruce is updated to Oliver and both lists refer to it

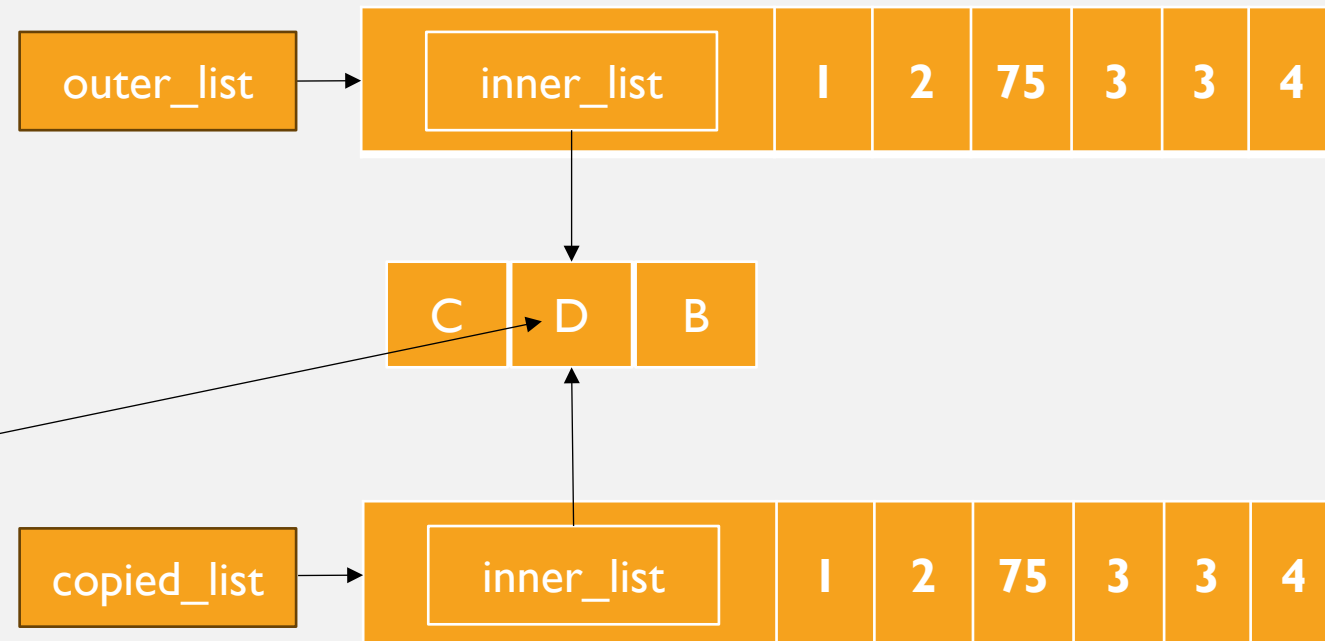
COPYING LISTS

```
01  # Ways to copy
02  # Aliasing
03  original_list = [1, 2, 75, 3, 3, 4]
04  copied_list_1 = original_list
05  # Yet another way
06  copied_list_2 = list(original_list)
07  # Specify range of slice - defaults are 0 and len(list)
08  copied_list_3 = original_list[:]
09  # Alternative - imported library – remember this one
10  import copy #imports a library with another copy method
11  copied_list_4 = copy.copy(original_list)
```

PROBLEM - SHALLOW COPY

- All of the above are ways of doing a *shallow copy* gives a listing containing a copy of all immutable values
- But when a list contains other objects, it is a pointer to a complex structure that is copied – not the data in the structure

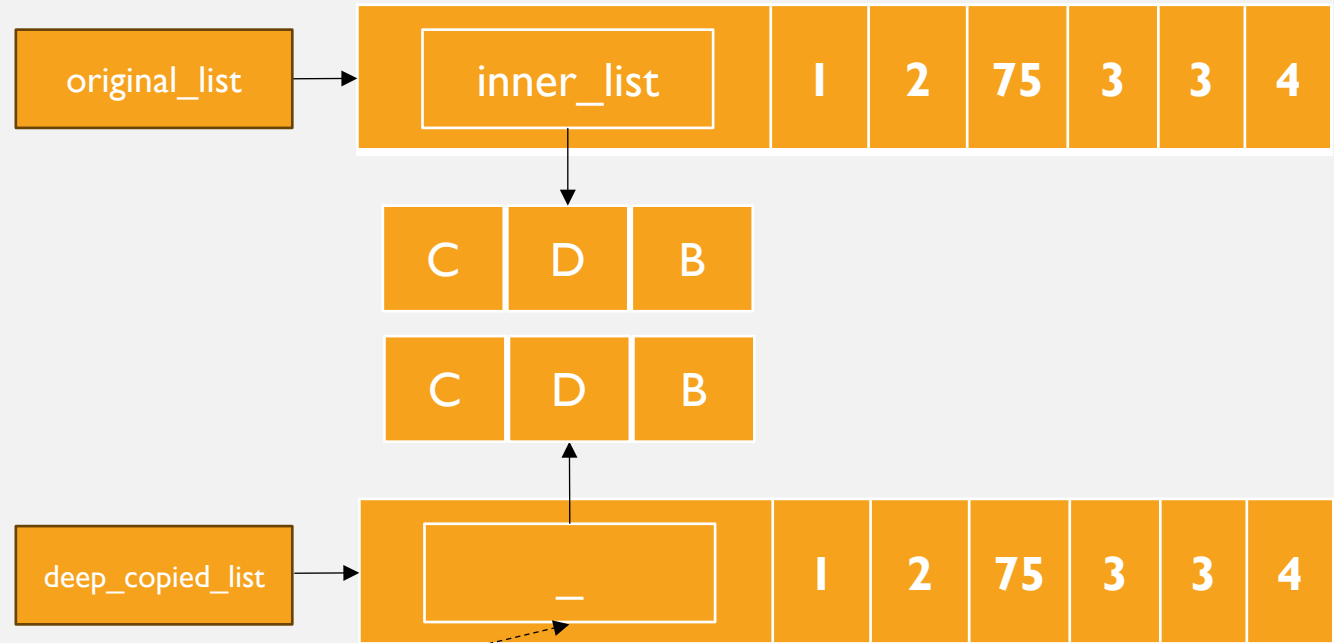
```
01 # first index position containing
02 # copy the list
03 outer_list = [1, 2, 75, 3, 3, 4]
04 inner_list = ["C", "D", "B"]
05 outer_list.insert(0, inner_list)
06 # outer_list -> [['C', 'D', 'B'], 1, 2, 75, 3, 3, 4]
07 copied_list = outer_list.copy()
08 # copied_list -> [['C', 'D', 'B'], 1, 2, 75, 3, 3, 4]
09 inner_list[1] = "O"
10 # outer_list -> [['C', 'O', 'B'], 1, 2, 75, 3, 3, 4]
11 # copied_list -> [['C', 'O', 'B'], 1, 2, 75, 3, 3, 4]
```



- In this case Line 09 updates the inner list referred to by both
- “D” is updated to “O” – but if we only want one of `outer_list` or `copied_list` to be updated we cannot do it.

DEEP COPY (I)

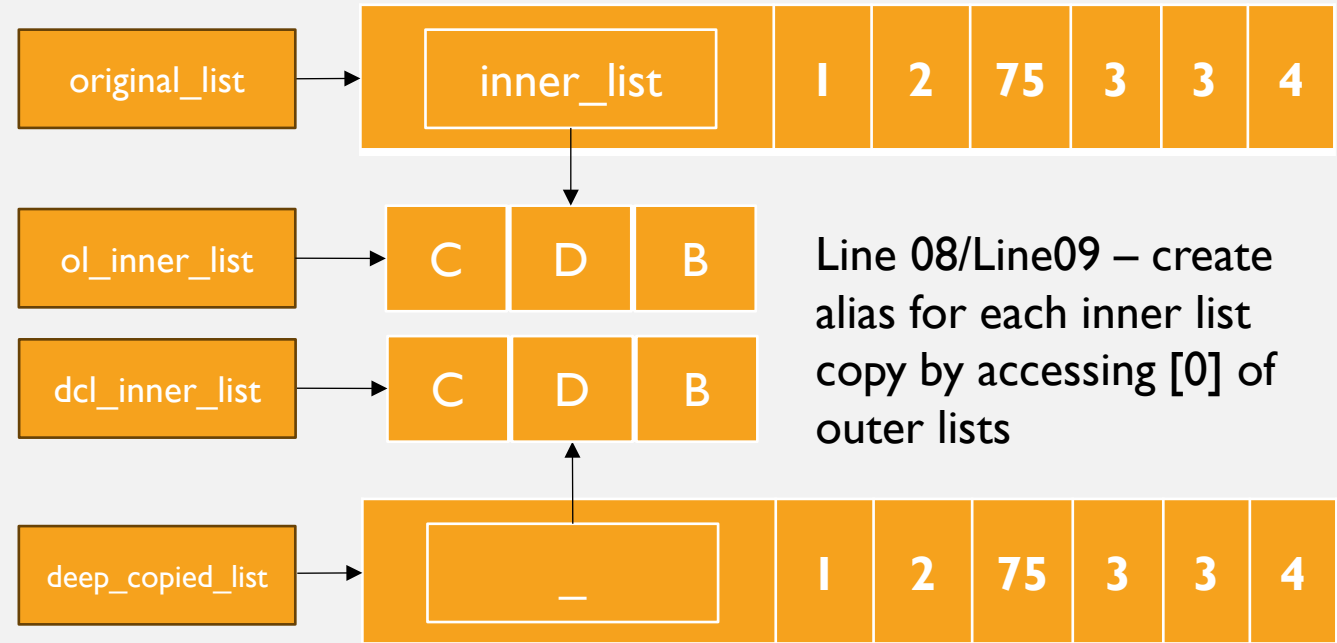
```
01 # Deep copy method
02 # Creates a full copy that can be manipulated independent of the original
03 import copy
04 original_list = [1, 2, 75, 3, 3, 4]
05 inner_list = ["C", "D", "B"]
06 original_list.insert(0, inner_list) #insert inner list
07 deep_copied_list = copy.deepcopy(original_list) #deep copy by method
```



Unnamed copy of inner list

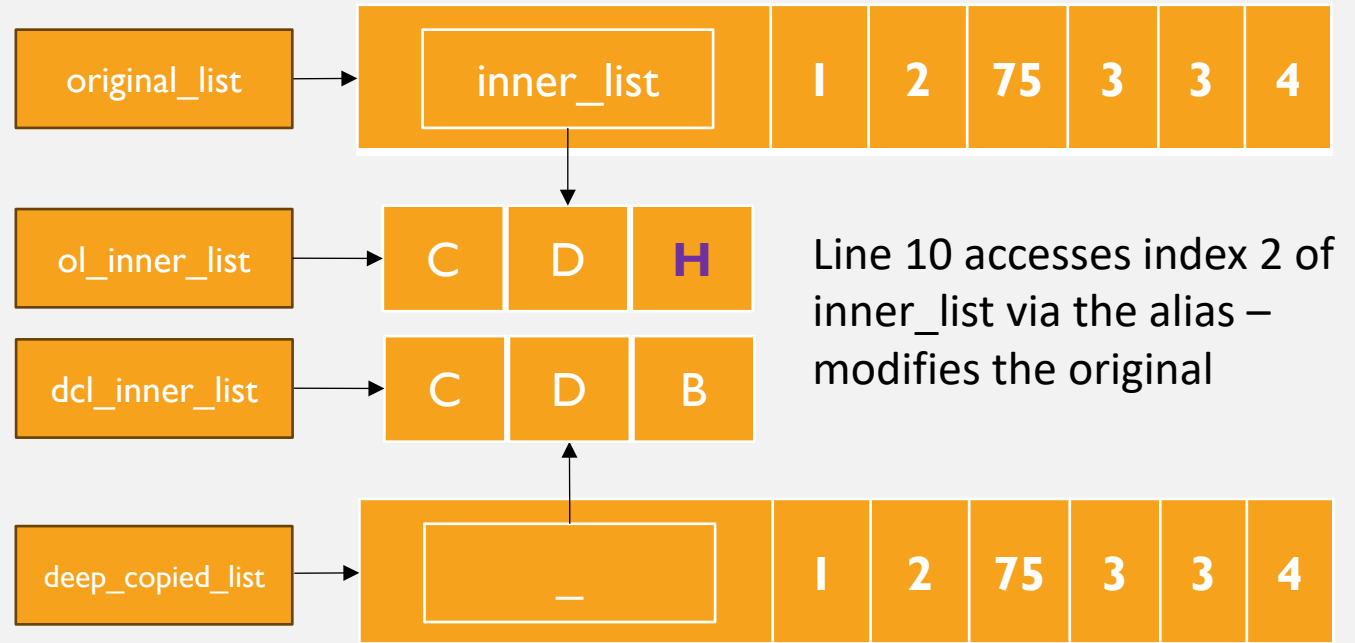
DEEP COPY (2)

```
01 # Deep copy method
02 # Creates a full copy that can be manipulated independent of the original
03 import copy
04 original_list = [1, 2, 75, 3, 3, 4]
05 inner_list = ["C", "D", "B"]
06 original_list.insert(0, inner_list) #insert inner list
07 deep_copied_list = copy.deepcopy(original_list) #deep copy by method
08 ol_inner_list = original_list[0] # get the inner list from original
09 dcl_inner_list = deep_copied_list[0] #get the inner list from deep copy
```



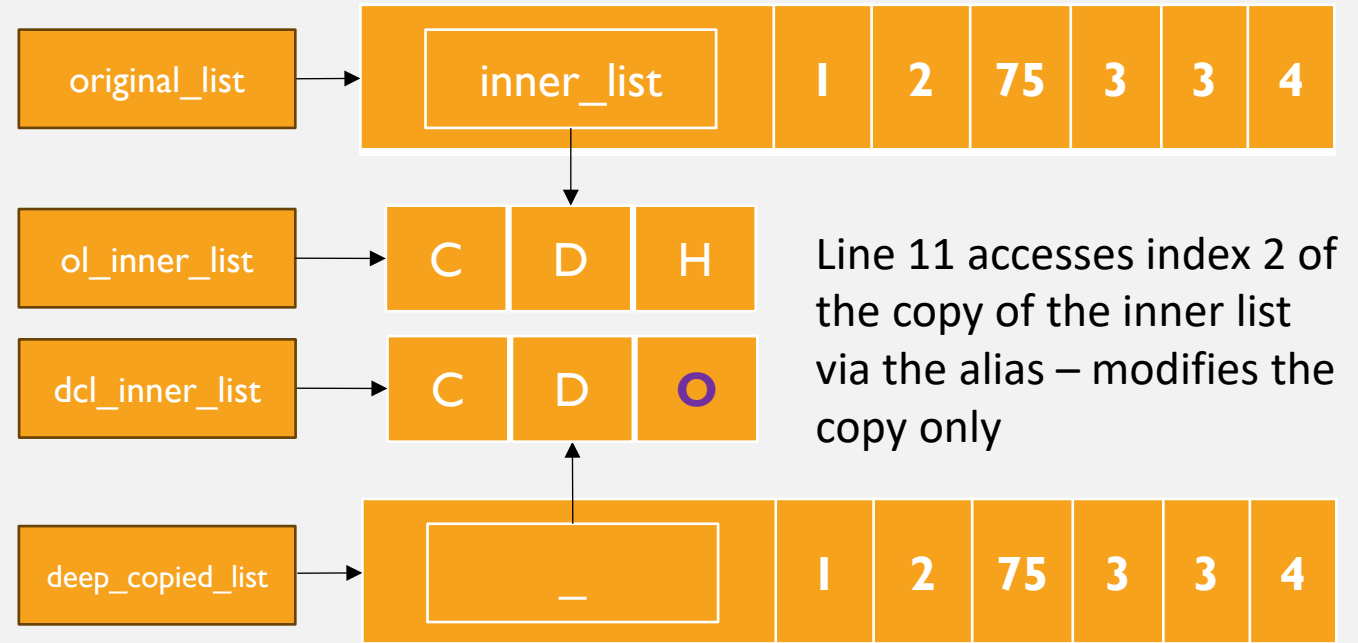
DEEP COPY(3)

```
01 # Deep copy method
02 # Creates a full copy that can be manipulated independent of the original
03 import copy
04 original_list = [1, 2, 75, 3, 3, 4]
05 inner_list = ["C", "D", "B"]
06 original_list.insert(0, inner_list) #insert inner list
07 deep_copied_list = copy.deepcopy(original_list) #deep copy by method
08 ol_inner_list = original_list[0] # get the inner list from original
09 dcl_inner_list = deep_copied_list[0] #get the inner list from deep copy
10 ol_inner_list[2] = "H" # Update the original inner list via alias
```



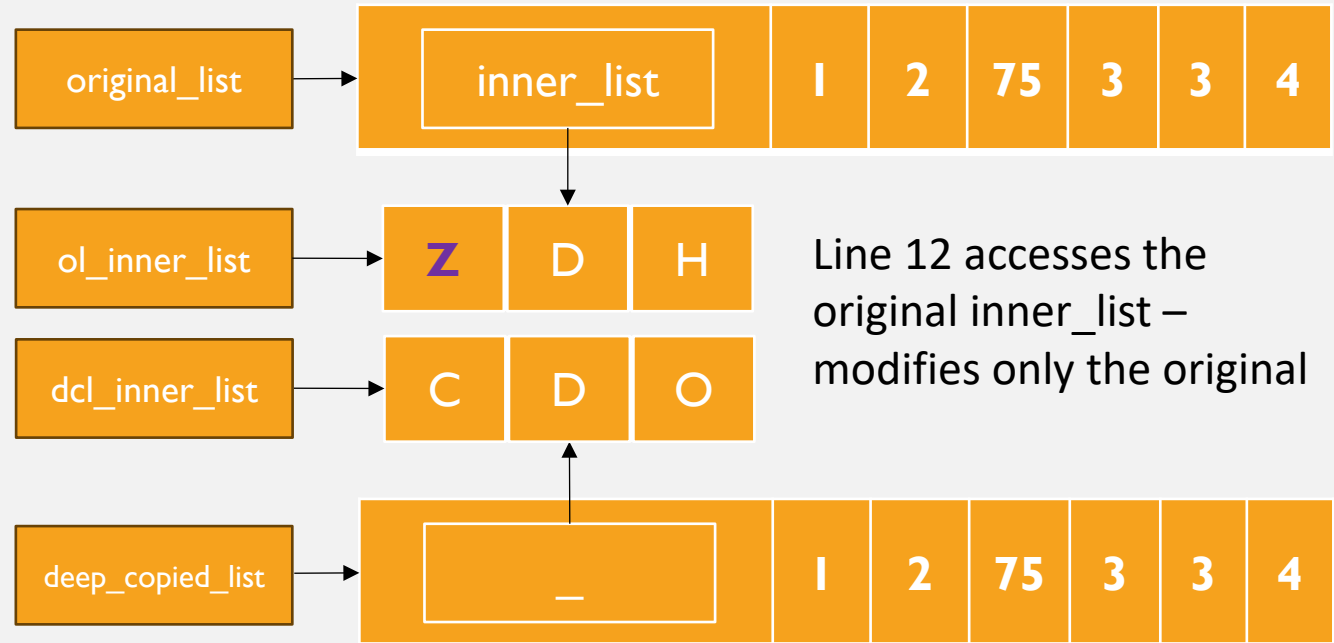
DEEP COPY (4)

```
01 # Deep copy method
02 # Creates a full copy that can be manipulated independent of the original
03 import copy
04 original_list = [1, 2, 75, 3, 3, 4]
05 inner_list = ["C", "D", "B"]
06 original_list.insert(0, inner_list) #insert inner list
07 deep_copied_list = copy.deepcopy(original_list) #deep copy by method
08 ol_inner_list = original_list[0] # get the inner list from original
09 dcl_inner_list = deep_copied_list[0] #get the inner list from deep copy
10 ol_inner_list[2] = "H" # Update the original inner list via alias
11 dcl_inner_list[2] = "O" # Update the copy inner list via alias
```



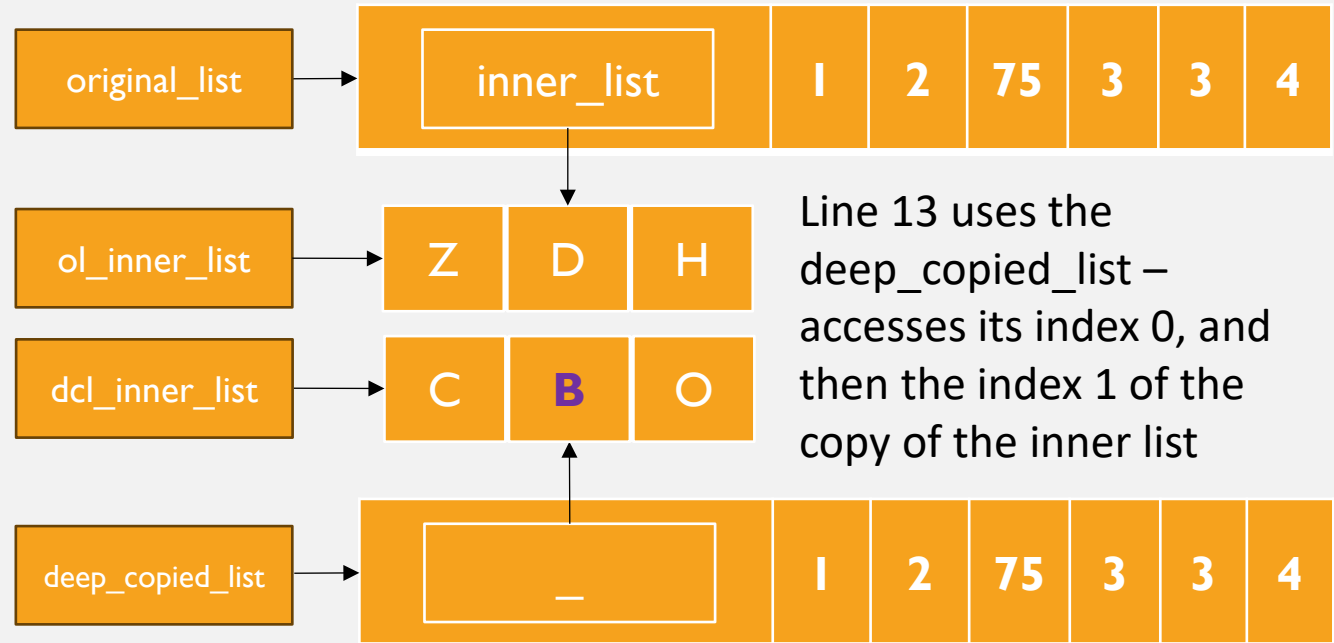
DEEP COPY (5)

```
01 # Deep copy method
02 # Creates a full copy that can be manipulated independent of the original
03 import copy
04 original_list = [1, 2, 75, 3, 3, 4]
05 inner_list = ["C", "D", "B"]
06 original_list.insert(0, inner_list) #insert inner list
07 deep_copied_list = copy.deepcopy(original_list) #deep copy by method
08 ol_inner_list = original_list[0] # get the inner list from original
09 dcl_inner_list = deep_copied_list[0] #get the inner list from deep copy
10 ol_inner_list[2] = "H" # Update the original inner list via alias
11 dcl_inner_list[2] = "O" # Update the copy inner list via alias
12 inner_list[0] = "Z" # update the original inner list
```



DEEP COPY (5)

```
01 # Deep copy method
02 # Creates a full copy that can be manipulated independent of the original
03 import copy
04 original_list = [1, 2, 75, 3, 3, 4]
05 inner_list = ["C", "D", "B"]
06 original_list.insert(0, inner_list) #insert inner list
07 deep_copied_list = copy.deepcopy(original_list) #deep copy by method
08 ol_inner_list = original_list[0] # get the inner list from original
09 dcl_inner_list = deep_copied_list[0] #get the inner list from deep copy
10 ol_inner_list[2] = "H" # Update the original inner list via alias
11 dcl_inner_list[2] = "O" # Update the copy inner list via alias
12 inner_list[0] = "Z" # update the original inner list
13 deep_copied_list[0][1] = "B" # update copy inner list via indexing
```



ACCESSING 2-DIMENSIONAL LISTS

- Recall strings:

```
01 # Recall Strings
02 name = "Stephanie"
03 # Offset [3] accesses string "p"
04 print(name[3]) # p
05 # Offset [0] is applied to string "p"
06 print(name[3][0]) # p
07 # Similar for lists
08 # Offset [0] accesses the pointer to the list
09 print(deep_copied_list[0]) # ['C', 'B', 'O']
10 # Offset [1] is applied to the list accessed
11 print(deep_copied_list[0][1]) # B
```

ITERATING THROUGH LISTS

Use the for loop to
iterate over each
item in a list

```
01 # Iterating
02 # Can iterate the same way
03 my_list = [1, 2, 75, 3, 3, 4]
04
05 # Access by value - no access to index information
06 for item in my_list:
07     # use of second parameter to remove the default \n
08     print(item, end=" ")
09 print() # used to print a single newline at end of loop
10
11 # Access by index - access to both value and index
12 for i in range(len(my_list)):
13     print(my_list[i], end=" ")
14 print()
```

+ ON LISTS

- You can concatenate lists together using the + operator (just like strings)

```
01 # Concatenating Two lists
02 names_1 = ["Clark", "Diana", "Bruce"]
03 names_2 = ["Conner", "Donna", "Tim"]
04 all_names = names_1 + names_2
05 print(all_names) # all immutable types
06 #['Clark', 'Diana', 'Bruce', 'Conner', 'Donna', 'Tim']
```

- Performs a shallow copy – so need to be careful if we are going to use the original variables

```
07 names_2[2] = ["Richard", "Tim", "Damien"] # make names 2 have a nested list
08 all_names = names_1 + names_2 # concatenate again
09 print(all_names)
10 # ['Clark', 'Diana', 'Bruce', 'Conner', 'Donna', ['Richard', 'Tim', 'Damien']]
11 names_2[2][2] = "Stephanie" # update last item of inner list
12 print(all_names)
13 # ['Clark', 'Diana', 'Bruce', 'Conner', 'Donna', ['Richard', 'Tim', 'Stephanie']]
```

UNPACKING OPERATOR *

- The elements of a list can be unpacked into their single element variables using the * operator

```
01 # Unpacking Nested Lists
02 team_1 = ["Clark", "Diana", "Bruce"]
03 team_2 = ["Conner", "Donna", "Tim"]
04 # Create 2 nested lists inside a list
05 teams = []
06 teams.append(team_1)
07 teams.append(team_2)
08 print(teams) # [['Clark', 'Diana', 'Bruce'], ['Conner', 'Donna', 'Tim']]
09 team_up = [*team_1, *team_2] # Unpack the nested lists inside a new list
10 print(team_up) # ['Clark', 'Diana', 'Bruce', 'Conner', 'Donna', 'Tim']
11 # Unpack the nested lists and insert some new items
12 big_team_up = ["Oliver", *team_1, "Dinah", *team_2]
13 print(big_team_up) # ['Oliver', 'Clark', 'Diana', 'Bruce', 'Dinah', 'Conner', 'Donna', 'Tim']
```

SEARCHING WITHIN LISTS

- Challenge using `.index()` – if the string is not in the list it, it fails

```
1 # Membership in lists
2 # Concatenating Two lists
3 names_1 = ["Clark", "Diana", "Bruce"]
4 names_2 = ["Conner", "Donna", "Tim"]
5 # Alternate - Index searching
6 names_1.index("Conner")
```

ValueError Traceback (most recent call last)

Cell In[51], line 12

```
9 print("Up Up and Away")

11 # Alternate - Index searching

--> 12 names_1.index("Conner")
```

ValueError: list.index(x): x not in list

- Alternative is to use the `in` keyword

```
1 if "Clark" in names_1:
2     print("Hi Clark!")
3 else:
4     print("Where did he go?")
```


HELPFUL FUNCTIONS

For a given list S:

- `max(S)`: returns the largest item in the sequence;
- `min(S)`: returns the smallest item in the sequence;
- `sum(S)`: returns the sum of the items in the sequence
 - Sequence needs to be something that can have a sum calculated
- `len(S)`: returns the number of items in the sequence.

PASSING MUTABLE OBJECTS TO FUNCTIONS

- When lists (and other mutable objects) are passed into functions, the reference (pointer) is passed — not a copy of the data
- This is the same issue as copying lists: both variables refer to the same underlying object
- Modifying the list inside a function will also modify the original list in the program
- This differs from immutable atomic types (e.g., integers), which cannot be modified in place – if you try to modify a parameter the value in the main program remains the same

MUTABLE OBJECTS IN FUNCTIONS EXAMPLE

```
01 # Mutable Objects in Functions
02 def add_team_member(team, new_member):
03     """ Adds a team member if they are not on the team.
04     :param team: the list of team members
05     :param add: the person to add to the team
06     :return: true if added, else false
07     """
08     if new_member not in team:
09         team.append(new_member)
10         return True
11     else:
12         return False
13
14 original_team = ["Diana", "Arthur", "John", "Harold", "Barry"]
15 print(original_team) # ['Diana', 'Arthur', 'John', 'Harold', 'Barry']
16
17 add_team_member(original_team, "Clark")
18 add_team_member(original_team, "Bruce")
19 print(original_team) # ['Diana', 'Arthur', 'John', 'Harold', 'Barry', 'Clark', 'Bruce']
```

IN CLASS EXAMPLE: DECK OF CARDS

- Create a list containing all possible the cards
 - Each card is a string
 - Example: “J♥”
- How do I create a single card like the one above?

```
01 # Creating a deck of cards
02 # highest to lowest rankings
03 suit = ["♠", "♥", "♦", "♣"]
04 rank = ["A", "K", "Q", "J", "10", "9", "8", "7", "6", "5", "4", "3", "2"]
```

IN CLASS EXAMPLE: DECK OF CARDS

- Create a list containing all possible the cards
 - Each card is a string
 - Example: “J♥”
- How do I create a single card like the one above?
 - Concatenate operation using the indices
- How do I create the deck?

```
01 # Creating a deck of cards
02 # highest to lowest rankings
03 suit = ["♠", "♥", "♦", "♣"]
04 rank = ["A", "K", "Q", "J", "10", "9", "8", "7", "6", "5", "4", "3", "2"]
05 #10 of hearts
06 ten_hearts = rank[4] + suit[1]
07 print(f"10 of hearts: {ten_hearts}")
```

IN CLASS EXAMPLE: DECK OF CARDS

- Create a list containing all possible the cards
 - Each card is a string
 - Example: "J♥"
- How do I create a single card like the one above?
 - Concatenate operation using the indices
- How do I create the deck?
 - Loop through each list and append a concatenation for each combination

```
01 # Creating a deck of cards
02 # highest to lowest rankings
03 suit = ["♠", "♥", "♦", "♣"]
04 rank = ["A", "K", "Q", "J", "10", "9", "8", "7", "6", "5", "4", "3", "2"]
05 #10 of hearts
06 ten_hearts = rank[4] + suit[1]
07 print(f"10 of hearts: {ten_hearts}")
08 # create a deck of cards with all cards grouped by suit and ordered by rank
09 deck = []
10 # using value based for loop as I do not need the index
11 for s in suit:
12     for card in rank:
13         # add it to the end of the deck
14         deck.append(card + s)
15 print(deck) # full deck
```

IN CLASS EXAMPLE: SWAP CARDS

- Suppose we want to swap a card with another one, what do we do?

```
01  # Swapping Places
02  # Get index of first (and only) 2 of hearts
03  idx_2h = deck.index("2♥")
04  # Get index of first (and only) 2 of clubs
05  idx_2c = deck.index("2♣")
06  # Key pattern - Holding temporary variable
07  # Swap method 2
08  temp_card = deck[idx_2h] # move card 1 to temp variable
09  deck[idx_2h] = deck[idx_2c] # move card 2 to card 1
10  deck[idx_2c] = temp_card # move temp to card 2
11  print(deck)
12  # Swap back
13  temp_card = deck[idx_2c] # move card 1 to temp variable
14  deck[idx_2c] = deck[idx_2h] # move card 2 to card 1
15  deck[idx_2h] = temp_card # move temp to card 2
16  print(deck)
17  #Python Specific Swap Expression
18  deck[idx_2h], deck[idx_2c] = deck[idx_2c], deck[idx_2h]
19  print(deck)
```

FIND A PARTNER

- Given a list of integers find the median value
- If we want to do this in a function:
 - What are the parameters?
 - What is the return type?
 - What is our function called?
- Questions to start?
 - What more do you need to know about the list?
 - What if the list has an even number of items?
 - What do we do if the list has strings, or Booleans or other things?

TRY AT HOME

- Write a function to determine how many unique items are in a list
- Write a function that takes as input parameters 2 lists and returns a single list containing their combined contents
 - I would do this a couple of different ways to learn the variety of options you have available.
- Write a function called `contains_duplicates` that takes as an argument a list and returns `True` if the list contains any duplicates, or `False` otherwise
- Write a function called `sub_search` that takes as arguments two lists (`lst` and `sub_lst`) and returns the smallest index position of a sub list within a list (or `-1` if not found)

TRY AT HOME

- Given a list of classroom capacities (integers) and a list of course enrolment numbers, where the courses are all scheduled at the same time.
- Determine if there are any courses that cannot be accommodated.
- To accommodate a course any classroom must have enough capacity (seats)
- For example:
 - Capacities = [4, 10, 5] (a classroom that holds 4 students, another for 10 students and another for 5 students)
 - Enrolments = [7, 1, 2, 3] (a course with 7 students, 1 student, 2 students and 3 students)
 - Then one of the courses cannot be accommodated
 - Advanced answer: the least students that could possibly be inconvenienced is 1